

WEEK 12

Question 1

Create multiple threads and demonstrate different thread priorities using *MIN_PRIORITY*, *NORM_PRIORITY*, and *MAX_PRIORITY*

Java Code:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println(getName() + " priority: " + getPriority());  
    }  
}  
  
public class Lab1 {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        MyThread t2 = new MyThread();  
        MyThread t3 = new MyThread();  
  
        t1.setPriority(Thread.MIN_PRIORITY);  
        t2.setPriority(Thread.NORM_PRIORITY);  
        t3.setPriority(Thread.MAX_PRIORITY);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab1>javac Lab1.java  
C:\JavaLab\Lab1>java Lab1  
Thread-2 priority: 10  
Thread-1 priority: 5  
Thread-0 priority: 1  
C:\JavaLab\Lab1>
```

Question 2

Create three threads with different priorities and observe their execution order

Java Code:

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }
    public void run() {
        System.out.println(getName() + " (priority " + getPriority() + ") is running");
    }
}

public class Lab2 {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread A");
        MyThread t2 = new MyThread("Thread B");
        MyThread t3 = new MyThread("Thread C");

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab2>javac Lab2.java
C:\JavaLab\Lab2>java Lab2
Thread C (priority 10) is running
Thread A (priority 1) is running
Thread B (priority 5) is running
C:\JavaLab\Lab2>
```

Question 3

Demonstrate synchronization using a synchronized method for accessing a shared counter

Java Code:

```
class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}

class MyThread extends Thread {
    Counter counter;
    MyThread(Counter c) {
        counter = c;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    }
}

public class Lab3 {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        MyThread t1 = new MyThread(counter);
        MyThread t2 = new MyThread(counter);

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final count: " + counter.count);
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab3>javac Lab3.java
C:\JavaLab\Lab3>java Lab3
Final count: 2000
C:\JavaLab\Lab3>
```

Question 4

Demonstrate synchronization using a synchronized block

Java Code:

```
class Counter {
    int count = 0;

    void increment() {
        synchronized (this) {
            count++;
        }
    }
}

class MyThread extends Thread {
    Counter counter;
    MyThread(Counter c) {
        counter = c;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    }
}

public class Lab4 {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        MyThread t1 = new MyThread(counter);
        MyThread t2 = new MyThread(counter);

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println("Final count: " + counter.count);
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab4>javac Lab4.java
C:\JavaLab\Lab4>java Lab4
Final count: 2000
C:\JavaLab\Lab4>
```

Question 5

Demonstrate a race condition and eliminate it using synchronization

Java Code:

```
class Counter {
    int count = 0;

    void increment() {
        count++; // not synchronized, may cause race condition
    }

    synchronized void safeIncrement() {
        count++; // synchronized, no race condition
    }
}

class MyThread extends Thread {
    Counter counter;
    boolean safe;
    MyThread(Counter c, boolean safe) {
        counter = c;
        this.safe = safe;
    }
    public void run() {
        for (int i = 0; i < 1000000; i++) {
            if (safe) {
                counter.safeIncrement();
            } else {
                counter.increment();
            }
        }
    }
}

public class Lab5 {
    public static void main(String[] args) throws InterruptedException {
        // Without synchronization
        Counter c1 = new Counter();
        MyThread a1 = new MyThread(c1, false);
        MyThread a2 = new MyThread(c1, false);
        a1.start(); a2.start();
        a1.join(); a2.join();
        System.out.println("Without synchronization, count = " + c1.count);

        // With synchronization
        Counter c2 = new Counter();
        MyThread b1 = new MyThread(c2, true);
        MyThread b2 = new MyThread(c2, true);
        b1.start(); b2.start();
        b1.join(); b2.join();
        System.out.println("With synchronization, count = " + c2.count);
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab5>javac Lab5.java
C:\JavaLab\Lab5>java Lab5
Without synchronization, count = 1941166
With synchronization, count = 2000000
C:\JavaLab\Lab5>
```

Question 6

Implement a synchronized ticket-booking system where multiple threads attempt to book tickets simultaneously

Java Code:

```
class TicketCounter {
    int tickets = 3;

    synchronized void bookTicket(String name) {
        if (tickets > 0) {
            tickets--;
            System.out.println(name + " booked a ticket. Remaining: " + tickets);
        } else {
            System.out.println(name + " could not book a ticket.");
        }
    }
}

class Customer extends Thread {
    TicketCounter counter;
    String name;
    Customer(TicketCounter c, String name) {
        counter = c;
        this.name = name;
    }
    public void run() {
        counter.bookTicket(name);
    }
}

public class Lab6 {
    public static void main(String[] args) throws InterruptedException {
        TicketCounter counter = new TicketCounter();

        Customer c1 = new Customer(counter, "Customer 1");
        Customer c2 = new Customer(counter, "Customer 2");
        Customer c3 = new Customer(counter, "Customer 3");
        Customer c4 = new Customer(counter, "Customer 4");

        c1.start();
        c2.start();
        c3.start();
        c4.start();

        c1.join();
        c2.join();
        c3.join();
        c4.join();
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab6>javac Lab6.java
C:\JavaLab\Lab6>java Lab6
Customer 1 booked a ticket. Remaining: 2
Customer 4 booked a ticket. Remaining: 1
Customer 3 booked a ticket. Remaining: 0
Customer 2 could not book a ticket.
C:\JavaLab\Lab6>
```

Question 7

Introduce lambda expressions by implementing a functional interface for addition, subtraction, multiplication, and division

Java Code:

```
interface Operation {
    int calculate(int a, int b);
}

public class Lab7 {
    public static void main(String[] args) {
        Operation add = (a, b) -> a + b;
        Operation sub = (a, b) -> a - b;
        Operation mul = (a, b) -> a * b;
        Operation div = (a, b) -> a / b;

        System.out.println("Addition: " + add.calculate(10, 5));
        System.out.println("Subtraction: " + sub.calculate(10, 5));
        System.out.println("Multiplication: " + mul.calculate(10, 5));
        System.out.println("Division: " + div.calculate(10, 5));
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab7>javac Lab7.java
C:\JavaLab\Lab7>java Lab7
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
C:\JavaLab\Lab7>
```

Question 8

Demonstrate a lambda expression with a single parameter and multiple parameters

Java Code:

```
interface Square {  
    int find(int a);  
}  
  
interface Add {  
    int find(int a, int b);  
}  
  
public class Lab8 {  
    public static void main(String[] args) {  
        Square sq = a -> a * a;  
        Add add = (a, b) -> a + b;  
  
        System.out.println("Square: " + sq.find(5));  
        System.out.println("Sum: " + add.find(5, 10));  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab8>javac Lab8.java  
C:\JavaLab\Lab8>java Lab8  
Square: 25  
Sum: 15  
C:\JavaLab\Lab8>
```

Question 9

Demonstrate a block lambda expression containing multiple statements

Java Code:

```
interface Check {  
    void test(int n);  
}  
  
public class Lab9 {  
    public static void main(String[] args) {  
        Check check = (n) -> {  
            System.out.println("Number: " + n);  
            if (n % 2 == 0) {  
                System.out.println("Even");  
            } else {  
                System.out.println("Odd");  
            }  
        };  
  
        check.test(7);  
        check.test(10);  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab9>javac Lab9.java  
C:\JavaLab\Lab9>java Lab9  
Number: 7  
Odd  
Number: 10  
Even  
C:\JavaLab\Lab9>
```

Question 10

Use a block lambda expression to find the largest of two numbers and return the result

Java Code:

```
interface Max {
    int find(int a, int b);
}

public class Lab10 {
    public static void main(String[] args) {
        Max max = (a, b) -> {
            if (a > b) {
                return a;
            } else {
                return b;
            }
        };

        System.out.println("Largest: " + max.find(20, 35));
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab10>javac Lab10.java
C:\JavaLab\Lab10>java Lab10
Largest: 35
C:\JavaLab\Lab10>
```

Question 11

Create a generic functional interface and implement it using lambda expressions for different data types

Java Code:

```
interface Show<T> {  
    void display(T value);  
}  
  
public class Lab11 {  
    public static void main(String[] args) {  
        Show<Integer> showInt = n -> System.out.println("Integer: " + n);  
        Show<String> showStr = s -> System.out.println("String: " + s);  
  
        showInt.display(100);  
        showStr.display("Hello");  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab11>javac Lab11.java  
C:\JavaLab\Lab11>java Lab11  
Integer: 100  
String: Hello  
C:\JavaLab\Lab11>
```

Question 12

Create a generic functional interface that accepts two values and performs an operation using a lambda expression

Java Code:

```
interface Operation<T> {  
    T apply(T a, T b);  
}  
  
public class Lab12 {  
    public static void main(String[] args) {  
        Operation<Integer> add = (a, b) -> a + b;  
        Operation<String> concat = (a, b) -> a + b;  
  
        System.out.println("Sum: " + add.apply(10, 20));  
        System.out.println("Concat: " + concat.apply("Hello ", "World"));  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab12>javac Lab12.java  
C:\JavaLab\Lab12>java Lab12  
Sum: 30  
Concat: Hello World  
C:\JavaLab\Lab12>
```

Question 13

Use lambda expressions to display and filter elements of an ArrayList

Java Code:

```
import java.util.ArrayList;

public class Lab13 {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);
        list.add(5);

        System.out.println("All elements:");
        list.forEach(n -> System.out.println(n));

        System.out.println("Even elements:");
        list.forEach(n -> {
            if (n % 2 == 0) {
                System.out.println(n);
            }
        });
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab13>javac Lab13.java
C:\JavaLab\Lab13>java Lab13
All elements:
1
2
3
4
5
Even elements:
2
4
C:\JavaLab\Lab13>
```

Question 14

Sort a list of integers and strings using lambda expressions

Java Code:

```
import java.util.ArrayList;

public class Lab14 {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);

        ArrayList<String> names = new ArrayList<>();
        names.add("Banana");
        names.add("Apple");
        names.add("Cherry");

        numbers.sort((a, b) -> a - b);
        names.sort((a, b) -> a.compareTo(b));

        System.out.println("Sorted numbers: " + numbers);
        System.out.println("Sorted names: " + names);
    }
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab14>javac Lab14.java
C:\JavaLab\Lab14>java Lab14
Sorted numbers: [10, 20, 30, 40]
Sorted names: [Apple, Banana, Cherry]
C:\JavaLab\Lab14>
```

Question 15

Create and execute a thread using a lambda expression

Java Code:

```
public class Lab15 {  
    public static void main(String[] args) {  
        Runnable task = () -> {  
            System.out.println("Thread is running using lambda expression");  
        };  
  
        Thread t = new Thread(task);  
        t.start();  
    }  
}
```

Output (Command Prompt):

```
C:\JavaLab\Lab15>javac Lab15.java  
C:\JavaLab\Lab15>java Lab15  
Thread is running using lambda expression  
C:\JavaLab\Lab15>
```